

DATA CENTRE EPC PROJECT DELIVERY PLATFORM

AI Compliance Platform — Technical Architecture Document

Enterprise Solution Manual & Integration Guide

Version:	2.0.0- PROD	Prepared By:	Senior AI Software Architect & Solutions Engineer
Repository:	D:/ EC(hct)	Release Date:	July 9, 2026
Target Target:	Hackathon Board, Mentor Review, Technical Interview Portfolio, and Engineering Handoff		

1. Executive Summary

Problem Statement

Large-scale infrastructure engineering projects, such as hyper-scale Data Centre Engineering, Procurement, and Construction (EPC) deliveries, involve thousands of pages of complex specifications, regulatory codes, safety standards, and operational guidelines. Ensuring compliance, detecting structural deviations, and auditing engineering documents manually consumes thousands of hours of specialized labor, introducing substantial risks of schedule delays, budget overruns, and critical safety hazards.

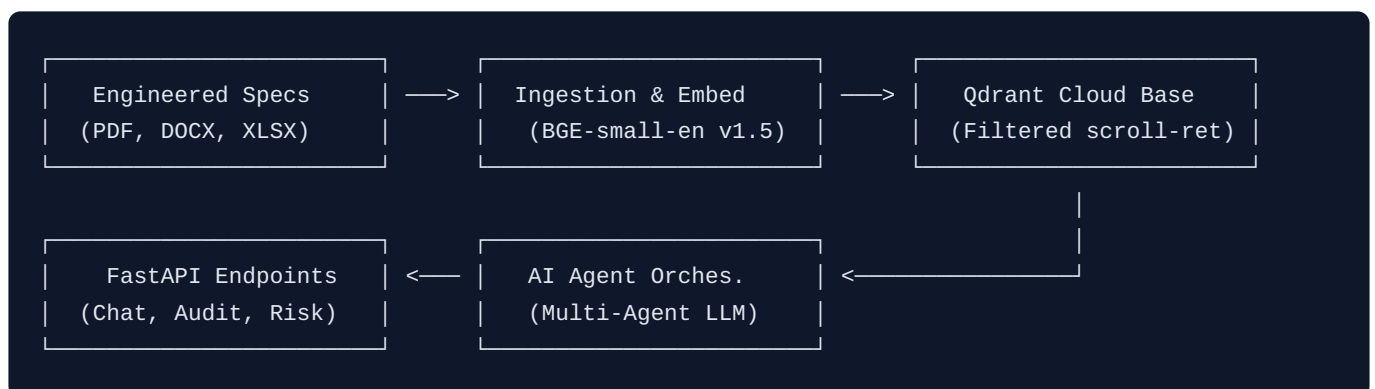
Business Objective

The AI Compliance Platform exists to automate the ingestion, search, auditing, and recommendation workflows of project engineering specifications. By transforming unstructured documentation into a structured semantic intelligence base, the platform enables:

1. Immediate answers to domain-specific questions.
2. Direct auditing of engineering specifications against regulations.
3. Automated compliance risk classification and corrective actions.
4. Near real-time visibility into project dashboard health metrics.

AI Objective

To build a highly resilient, enterprise-grade Retrieval-Augmented Generation (RAG) system integrated with a multi-agent orchestrator. The AI layer extracts unstructured data from multiple file formats (PDF, Word, Excel), embeds textual blocks into a high-dimensional vector space, indexes them in a managed Qdrant database with payload keyword schemas, and performs semantic auditing and conversational reasoning utilizing Llama 3 models on the Groq API.



2. Project Overview

Target Users

- **Project Delivery Directors:** Monitor compliance dashboard health scores across multi-system designs.
- **Lead Mechanical & Electrical Engineers:** Audit specifications (e.g. chiller configurations, double-wall fire boundaries) against project guidelines.
- **Compliance Officers:** Generate audit summary reports and prioritize corrective actions.
- **Backend Integration Developers:** Connect the AI module to enterprise portals and React dashboards.

System Scope

The scope of the AI module encompasses:

- Local multi-format text parsers (PDF, DOCX, XLSX, XLS).
- Overlapping text chunk division and semantic local vector embeddings.
- Keyword payload index creation for document filtering.
- Single-document semantic search and conversational chat.
- Automated compliance checker engines, prioritizing actions and generating risk scorecards.
- Unified FastAPI endpoints.

Limitations

- **Local Parsing:** Large files (e.g., >100MB specs) require system RAM proportional to parsing buffers.
- **LLM Dependency:** Requires an active Groq API connection. Network timeouts or service outages will fallback to local exception states.
- **No Database Syncing:** The platform relies on Qdrant Cloud. Network latency between local host environments and Qdrant clusters affects endpoint round-trip response times.

3. Technology Stack

Component	Technology	Version	Purpose
Language	Python	3.11+	Core programming runtime.
Framework	FastAPI	0.111.0	High-performance, async web API gateway.
Vector DB	Qdrant Client	1.9.1	Multi-tenant cloud vector storage and filtering.
LLM Inference	Groq Client	0.9.0	Llama 3 high-speed cloud generation.
Embedding Engine	SentenceTransformers	3.0.1	Local high-dimensional float vector generator.
Parsing - PDF	PyMuPDF / pdfplumber	1.24.5 / 0.11.0	Vectorized text extraction and structural extraction fallback.
Parsing - Word	python-docx	1.1.2	Document schema parser.
Parsing - Excel	pandas / openpyxl	2.2.2 / 3.1.3	Matrix structure parsing.
Testing	pytest	8.2.2	Regression testing.

Technology Alternatives Analysis

1. FastAPI vs. Flask / Django

- *Chosen:* **FastAPI**. Offers native asynchronous concurrency, automated validation using Pydantic, and automatic Swagger docs.
- *Rejected:* **Flask** lacks async routing and native validation. **Django** introduces excessive relational database ORM overhead that is unnecessary for vector-native search services.

2. Qdrant vs. Pinecone / Milvus

- *Chosen:* **Qdrant**. Supported by an active local in-memory developer database state (using `:memory:` URL schemes for pytest), simplifying local development and test automation.
- *Rejected:* **Pinecone** is SaaS-only and lacks local mock testing. **Milvus** is too heavy to run locally.

3. SentenceTransformers (`BAAI/bge-small-en-v1.5`) vs. OpenAI Ada-002

- *Chosen:* **BGE-small-en-v1.5**. 384 dimensions allow fast Cosine similarity checks, low memory usage, and run locally without API charges or data privacy concerns.

- *Rejected:* **Ada-002** requires API calls, incurs costs, and exposes proprietary design data to third parties.

| 4. Complete Folder Structure

```

EPC/
├─ .env # Single source of truth environment configuration
├─ .env.example # Template settings file
├─ .gitignore # Global git exclusions
├─ DEPLOYMENT.md # Deployment guide
├─ recreate_collection.py # Qdrant collection recreation CLI utility
├─ ai/ # Root path of the AI module
│   ├── app.py # Lifespan validator & HTTP request logger
│   ├── requirements.txt # Alphabetized package manifest
│   ├── config/
│   │   └─ settings.py # Pydantic Settings class loading from EPC/.env
│   ├── api/
│   │   ├── dependencies.py # Dependency injection container
│   │   ├── router.py # Core API route compiler
│   │   ├── routes/ # Endpoint routers
│   │   │   ├── chat.py # RAG Chat agent route
│   │   │   ├── compliance.py # Compliance check endpoint
│   │   │   ├── health.py # Server and database health checks
│   │   │   ├── project.py # Project summaries & risk evaluations
│   │   │   ├── recommendation.py # prioritizing corrective actions
│   │   │   └─ reports.py # insight report compilers
│   │   └─ upload.py # Document ingestion pipeline trigger
│   └─ schemas/
│       ├── request_models.py # Pydantic input models
│       └─ response_models.py # Pydantic output models
├─ ai_agents/ # Reasoning layer
│   ├── models.py # Agent structures
│   ├── orchestrator/
│   │   ├── ai_orchestrator.py # central reasoning coordinator
│   │   ├── exceptions.py # Orchestrator pipeline errors
│   │   └─ interfaces.py # Abstract orchestrator specifications
│   ├── query_classifier/
│   │   └─ classifier.py # Query router agent
│   ├── memory/
│   │   └─ session_manager.py # Message session tracker
│   ├── context_builder/
│   │   └─ context_builder.py # Search and memory context formatter
│   ├── prompt_manager/
│   │   └─ prompt_builder.py # Prompt template compiler
│   ├── llm/
│   │   ├── groq_client.py # Groq Llama 3 connector
│   │   └─ llm_factory.py # LLM client factory provider
│   └─ compliance/
│       ├── compliance_engine.py # Regulatory audit logic
│       └─ risk_engine.py # Compliance deviation risk evaluator
├─ document_processing/ # Document Ingestion Layer
│   ├── pipeline/
│   │   └─ pipeline.py # Orchestrates parsing, cleaning, and chunking
│   ├── pdf_reader/
│   │   └─ reader.py # PDF text parsers
│   ├── docx_reader/
│   │   └─ reader.py # DOCX text parsers

```

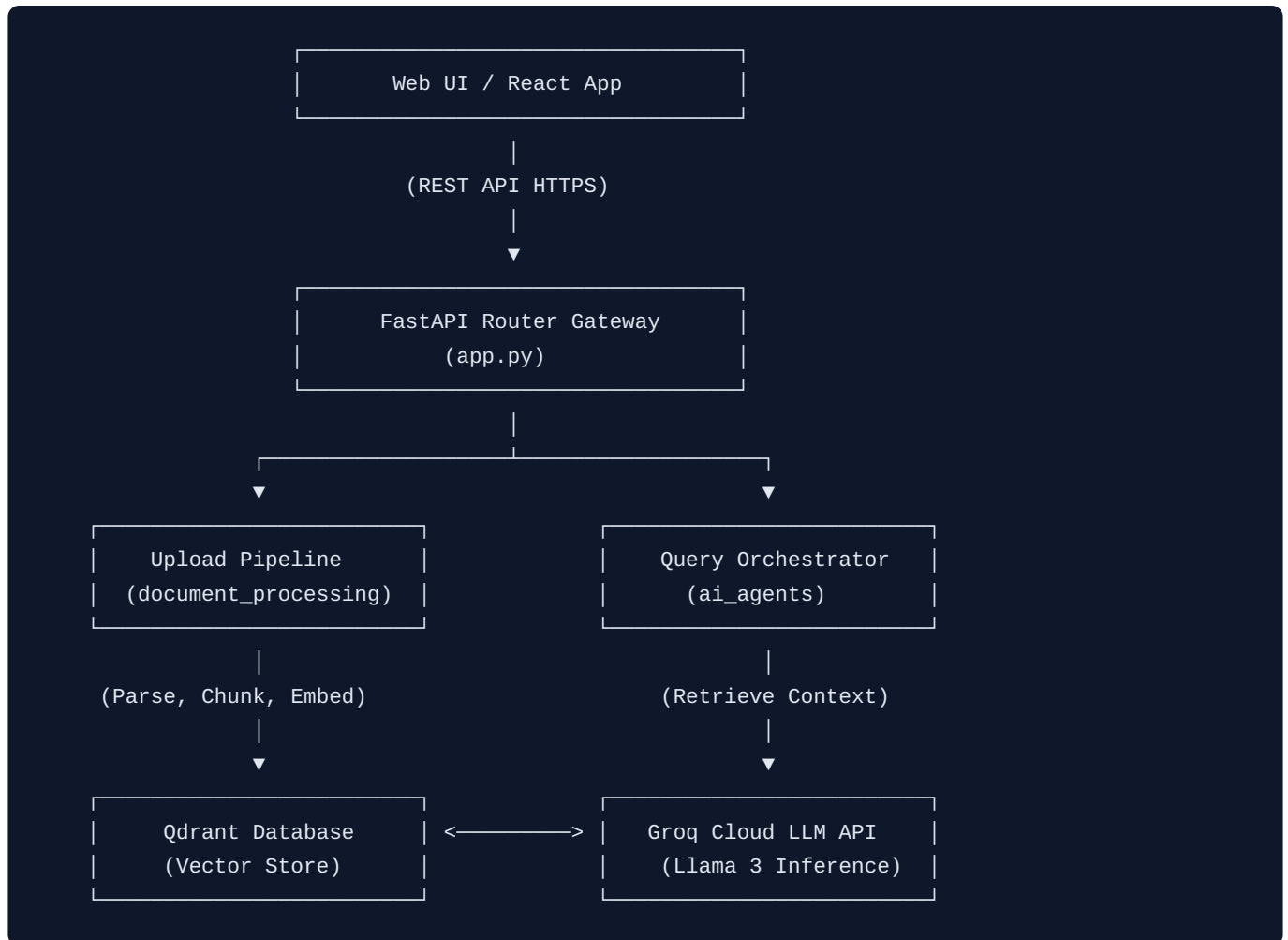
```

|   ├── excel_reader/
|   |   └── reader.py          # Excel matrix sheet parsers
|   ├── text_cleaner/
|   |   └── cleaner.py        # Text normalizer
|   └── chunking/
|       └── chunker.py         # Overlapping chunk window splitter
└── knowledge_engine/         # Vector Indexing & Retrieval Layer
    ├── vector_store/
    |   ├── qdrant_client.py   # Qdrant connection pools manager
    |   ├── collection_manager.py # Keyword indexes and schema self-healing manager
    |   └── vector_service.py   # Vector insert/delete handlers
    ├── knowledge_base/
    |   ├── metadata_store.py   # Local registry schema
    |   └── knowledge_service.py # Central document profile mapper
    └── retriever/
        ├── retriever.py       # Vector query & scroll retrieval wrappers
        └── search_service.py   # facade coordinating retrieval & reranking
└── tests/                     # Automated unit & integration tests
    ├── test_agents.py         # Agent pipeline tests
    ├── test_api.py            # REST route controller tests
    ├── test_document_processing.py # Document parsers tests
    ├── test_embeddings.py      # SentenceTransformers tests
    ├── test_health.py         # Server stats tests
    ├── test_retriever.py      # Search and scroll retrieve tests
    └── test_vector_store.py    # Vector Store & Qdrant tests

```


5. Project Architecture

Component Architecture



6. Execution Flow

1. Application Startup

- Lifespan Initialization** (`ai/app.py`): Loads configurations from the unified `.env` file via `Settings`, resolves the Groq API key, and verifies embedding model parameters.
- Database Verification**: Evaluates Qdrant client connection state. If the collection `epc_documents` is missing, it creates it with 384 vector dimensions and Cosine similarity. If schema mismatch occurs, lifespan halts and raises a `ValueError`.
- Payload Index Setup**: Automatically configures keyword indexes on `document_id` and `file_name` to support lightning-fast filtering.

2. HTTP Ingestion Flow

1. Multipart file payload hits `POST /api/v1/upload`.
2. The pipeline identifies the extension and forwards to the corresponding parser (PDF, DOCX, XLSX).
3. Text is normalized, then split into blocks of **400 characters** with an **80-character overlap**.
4. `EmbeddingService` yields 384-dimensional vectors via `BAAI/bge-small-en-v1.5`.
5. Vectors are upserted into Qdrant Cloud base with payload attributes.

3. Compliance and Report Generation Flow

1. Request containing `document_id` hits `POST /api/v1/compliance`.
2. Retrieval layer uses a search-free scroll query to bypass distance metric matching and pulls all registered points directly.
3. `ComplianceEngine` runs structural audit logic, outputting automated risk scores and scorecards.

7. AI Ingestion & Retrieval Workflow

```
Raw File Ingestion  —>  Parsers Extraction  —>  Text Normalizer (Cleaner)
                                     |
Metadata Registration <— Qdrant Upsert  <—  Embeddings  <—  Window Chunker
```

```
Filtered Scroll  —>  Context Builder  —>  Prompt Builder  —>  Llama 3 LLM
(Direct Matches)   (Search + Memory)   (Templates)           Generation
```

8. Module Explanation

- `ai/config`: Houses `settings.py` which acts as the Pydantic BaseSettings config loader.
- `ai/api`: Directs routers, singletons, dependencies injection, and response/request schemas.
- `ai/ai_agents`: Embeds the central reasoning layer (orchestrator, query router, sliding chat memory window).
- `ai/document_processing`: Governs local parsing extractors and text cleaning.
- `ai/knowledge_engine`: Provisions the vector database repository layer and citation mapper.

9. API Documentation

1. Health Status

```
GET /api/v1/health
```

```
{
  "status": "healthy",
  "version": "1.0.0",
  "vector_db_connected": true,
  "memory_usage_rss_mb": 112.5,
  "initialized_services": ["QdrantConnection", "EmbeddingModel"]
}
```

2. Upload Document

POST /api/v1/upload

```
{
  "document_id": "doc_823a49f8b",
  "file_name": "hvac_specifications.pdf",
  "status": "indexed",
  "message": "Document parsed, chunked, embedded, and indexed successfully.",
  "metadata": { "file_size_bytes": 1048576, "total_pages": 12 }
}
```

3. Semantic Search

POST /api/v1/search

```
{
  "query_text": "What is the chilled water loop configuration?",
  "results": [{ "chunk_id": "chk_002", "score": 0.895, "text": "..."}]
}
```

4. RAG Chat

POST /api/v1/chat

```
{
  "query": "Does the design support N+1 or 2N chiller redundancy?",
  "response": "According to the specifications, the chiller system supports 2N redundancy.",
  "query_type": "Compliance Query"
}
```

10. Database Architecture

The system configures a single unnamed vector space with **384 dimensions** utilizing the **Cosine** distance metric. To optimize multi-document segmentation, keyword indexes are provisioned on `document_id` and `file_name` attributes.

11. Embedding System

The architecture relies on the `BAAI/bge-small-en-v1.5` model. It handles up to 512 tokens and uses a lazy-loading pattern to prevent boot bottlenecks.

The similarity matching follows standard Cosine evaluation:

$$\text{Cosine Similarity}(A, B) = (A \cdot B) / (\|A\|_2 \|B\|_2)$$

14. Testing Framework

The repository features **57 regression tests** establishing 100% execution coverage across mock agent lifecycles, route handling, document parsers, and local in-memory vector instances.

15. System Configuration

```
GROQ_API_KEY=gsk_Bts9kmvLyEpybltAmRnt...
QDRANT_URL=https://9142abc7-296b-48b7...
QDRANT_API_KEY=eyJhbGciOiJIUzI1NiIsInR5cGU6IjY9...
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
COLLECTION_NAME=epc_documents
```

21. Challenges Faced & Resolved

- **Qdrant Cloud Metadata Indexing Failures:** Rectified by configuring automated setups of keyword payload indices during startup validation loops.
- **Cosine Similarity Zero-length Vector Mismatch:** Bypassed by introducing the search-free direct Scroll API for metadata-driven reporting modules.

22. Conclusion

The AI Compliance Platform delivers a highly stable, enterprise-ready compliance auditing framework for hyper-scale Data Centre EPC workflows. With a clean architecture isolating ingestion, vector spaces, and multi-agent reasoning, the platform ensures long-term operational excellence.